CM-EDU-002 / Control lab

CogniCar Path Tracking

Turn a reference path and the current vehicle state into a steering command. The student tracker uses pure pursuit with a cross-track correction, steering limits, and a steering-rate limit.

1 The interface

Python 3: one control step

```
steering_rad = tracker.step(path, pose, twist)
```

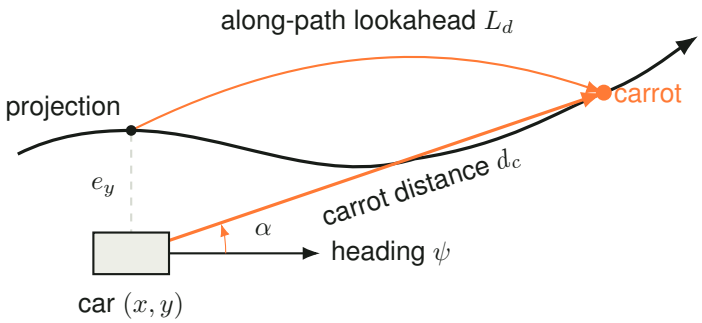
C++17: one control step

```
double steering_rad = tracker.step(path, pose, twist);
```

`path` is an ordered sequence of `PathPoint` values. `pose.position` and `pose.orientation` describe the car in the map frame; orientation is a quaternion. `twist.linear.x` is forward speed in the body frame, in m/s. The tracker extracts yaw in radians and returns steering in radians. The autopilot supplies the pose and twist from `LatestModel()`.

2 Project, look ahead, steer

Project the car onto the nearest path segment. From that projection, move forward along the ordered path to choose a target point (the *carrot*). Steer toward it.



Two different distances: L_d is measured along the path; d_c is the straight-line distance from the car to the carrot. Use d_c in the steering equation. The signed cross-track error e_y is positive to the left of the directed path.

> **Scope** This lab follows a known path in forward motion. It does not plan around obstacles or provide a reverse-driving controller. Path and vehicle pose must share a map frame.

3 The control law

1. Choose a lookahead

For forward speed $v \geq 0$, the supplied pipeline uses

$$L_d = \min(L_0 + k_v v,\ L_{\max}).$$

Here $L_0 > 0$ and $L_{\max} \geq L_0$ are in metres, and k_v is in seconds. A longer lookahead smooths corrections but can cut corners; a shorter one follows local geometry more closely but may oscillate. Negative speed needs a separate reverse-tracking policy; this formula alone does not enforce a positive floor in reverse.

2. Compute heading and lateral errors

For carrot (x_c, y_c) and vehicle yaw ψ:

$$\alpha = \mathrm{wrap}_{[-\pi,\pi]}\big(\mathrm{atan2}(y_c - y, x_c - x) - \psi\big), \qquad d_c = \max\big(\sqrt{(x_c - x)^2 + (y_c - y)^2}, 10^{-6}\big).$$

The small distance floor matches the reference helper and avoids a zero-distance singularity. Projection supplies e_y in metres.

3. Convert error to steering

The reference `steering_from_errors` helper implements

$$\boxed{\delta_{\mathrm{raw}} = k_p\,\mathrm{atan2}\big(2l\sin\alpha, d_c\big) - k_e e_y.}$$

l is the wheelbase in metres; k_p is the heading gain and k_e the lateral correction gain in rad/m. Positive steering turns left. The negative cross-track term steers back toward the path when the car is to its left.

4. Apply limits and remember the result

First clamp to `min_steering_rad` and `max_steering_rad`. Then restrict the change from the previous command:

$$|\delta_k - \delta_{k-1}| \leq \dot{\delta}_{\max}\,\Delta t.$$

Use a positive rate limit and time step. Set `dt_s` to the actual control period: the supplied autopilot runs at 100 Hz, so the student scaffold uses 0.01 s. Call `reset()` before a new run to clear the previous steering command.

> **Units throughout** Positions and distances: m. Speed: m/s. Yaw and steering: rad. Steering rate: rad/s. No degree conversion is needed at the car API or ROS steering topic.

4 Implement and check

In `exercises/1_path_tracking/`, edit `student_tracker.py` or `student_tracker.hpp`. Keep the fixed pipeline and replace one helper at a time, preserving its signature and return type.

Student helper	Responsibility
`project_to_path`	Closest projection and signed cross-track error.
`lookahead_point`	Carrot ahead of the projection on the path.
`heading_error`	Wrapped heading error and carrot distance; Python tuple, C++ `HeadingError`.
`steering_from_errors`	Raw steering from pursuit and lateral correction.
`clamp_steering`	Bound the steering angle in radians.
`rate_limit`	Bound the change per control step.

Run offline first

From a sourced SDK terminal:

Python 3

```
python3 exercises/1_path_tracking/check_tracker.py
```

C++17

```
cmake -S . -B build
cmake --build build --target check_tracker autopilot -j2
./build/check_tracker
```

After the check passes, start the simulator as described in *Getting Started*. Run `python3 exercises/autopilot.py --sim` or `./build/autopilot --sim`. The tracker controls steering; the low-level controller separately controls speed.

Check these cases

- Straight path, aligned car on the path: steering should be near zero.
- Car left or right of the path: check that the command corrects toward it.
- Heading crossing $-\pi/\pi$: wrap the difference to choose the shortest turn.
- Sharp turn or large offset: respect angle and rate limits; inspect recovery.
- Closed-path seam: check that the carrot advances through the loop correctly.

Tune one thing at a time

Start with the scaffold's supplied settings. Log cross-track error and steering at a fixed speed. If steering oscillates, try more lookahead or smaller gains. If the car cuts corners, reduce lookahead or speed and inspect the steering limits. Recheck each change offline and in simulation; an offline PASS alone does not validate a real-car run.

5 Your lab checklist

Work in `exercises/1_path_tracking/student_tracker.py` or `student_tracker.hpp`. These tasks follow the code's `IMPLEMENT THESE` section, in the same order.

Start with a working baseline

- [ ] Run the supplied `check_tracker` before editing. Record its RMS and worst cross-track error so you can compare your changes.

Complete the six student tasks

Keep each function's signature and return type. Leave the `FIXED PIPELINE` unchanged. After each task, rerun the offline checker, then try the tracker in the Digital Twin. Rebuild C++ after every edit.

- [ ] **`project_to_path`**: find the closest point on a path segment and return the signed cross-track error. Check offsets on both sides of the path.
- [ ] **`lookahead_point`**: move forward from the projection by L_d along the path. Check that the carrot continues across the closed-path seam.
- [ ] **`heading_error`**: return the wrapped heading error and straight-line carrot distance. Check the $-\pi/\pi$ boundary and zero distance.
- [ ] **`steering_from_errors`**: implement the pursuit term and cross-track correction. Check that positive steering turns left.
- [ ] **`clamp_steering`**: review or implement the angle limits. Try requests below, within and above the allowed range.
- [ ] **`rate_limit`**: limit the change from the previous command to $\dot{\delta}_{\max}\Delta t$. Check both increasing and decreasing requests.

Finish the lab and continue

- [ ] Run all the cases on the previous page. Save the final checker results and compare them with your baseline; do not rely on PASS alone.
- [ ] Run `autopilot --sim` using the commands on the previous page. Compare the reference and driven paths, cross-track error and steering.
- [ ] Change one lookahead setting or gain at a fixed speed. Save a before/after plot and explain what improved and what got worse.
- [ ] Review the real-car steps in *Getting Started* with your instructor. Confirm the map frame, fresh feedback and the lab's stop procedure before driving.

Next lab Continue with `exercises/3_low_level_control/student_low_level_control.py` or its `.hpp` counterpart to study how the autopilot controls speed. Start by running the matching `check_low_level_control` checker.